CS228-2026 Logic for Computer Science: Midsemester Exam
Maximum Marks: 70 | Maximum Time: 120 mins

---

1. (6=3*2 marks) For each of the statements below, state if they are True or False. Give a brief justification for each of your claims.

   general marking scheme: 0.5 for correct answer + 1.5 for right justification

   (a) Given two formulae φ and ψ, if $\varphi \vee \psi$ is valid, then both φ and ψ are valid.

   Solution

   False. Take $\varphi := p$, and $\psi := \neg p$. Clearly, $\varphi \vee \psi = p \vee \neg p = \top$ is valid, but neither φ, nor ψ is valid. Deduct **1 mark** if no counterexample is given for ψ and φ

   (b) If resolution on φ terminates (that is, no new resolvents can be generated by applying resolution on clauses in φ) without deriving the empty clause $\emptyset$, then φ is satisfiable.

   Solution

   True. If φ is unsatisfiable, then from the completeness of resolution, we get that resolution must terminate with an empty clause. Since φ is either satisfiable or unsatisfiable, the claim follows.
   Note: in case a student misinterpret the question, that resolution is only applied on the clauses of φ and not on the resolvents, we have been lenient and given marks. That is, when someone has answered the question as **False**, and given a proper justification for it.
   Give **0.5 mark** for justification if neither Completeness nor Contrapositive is mentioned.

   (c) If $F \vdash G$ is valid then $\neg G \vDash \neg F$ holds.

   Solution

   True. By the Soundness Theorem, if $F \vdash G$, then $F \vDash G$. This means that any truth assignment satisfying F must also satisfy G. Taking the contrapositive, if an assignment falsifies G (making $\neg G$ true), it cannot satisfy F (thus making $\neg F$ true). Therefore, $\neg G \vDash \neg F$ holds.
   If answered **True** then give upto **0.5 mark** for justification without any ND proof or law of contrapositive being mentioned.

2. (15=5*3 marks) Consider the following pairs:

   (a) $F_1 = \{p \to q,\ q \to r,\ \neg r\}$, $\quad G_1 = \neg p \wedge \neg q$

   (b) $F_2 = \{p \to q,\ q \to r,\ p\}$, $\quad G_2 = \neg r$

   (c) $F_3 = \{p \to q,\ q \to r\}$, $\quad G_3 = p \vee r$

   (d) $F_4 = \{p \to q,\ q \to r\}$, $\quad G_4 = (p \vee q) \to (q \wedge r)$

   (e) $F_5 = \{p \to q,\ q \to r\}$, $\quad G_5 = (p \vee r) \vee \neg q$

For each of the above pairs, F_i denotes the set of premises and G_i denotes the conclusion. Determine the logical relationship between F_i and G_i, for each $i \in \{1, \ldots 5\}$:

(i) If $F_i \vdash G_i$ is a valid sequent, provide a proof using **Natural Deduction**.

(ii) If $F_i \vdash \neg G_i$ is a valid sequent, provide a proof using **Resolution** to show that $F_i \cup \{G_i\}$ is unsatisfiable.

(iii) If **neither** of the above holds, provide a satisfying assignment and a falsifying assignment for $F_i \cup \{G_i\}$.

Solution

Category (i): $F_1 \vdash G_1$ is a valid sequent.
Proof using Natural Deduction:

1.	$p \to q$	premise
2.	$q \to r$	premise
3.	$\neg r$	premise
4.	$\neg q$	MT 2, 3
5.	$\neg p$	MT 1, 4
6.	$\neg p \wedge \neg q$	$\wedge$i 5,4

Marking scheme:

- 3 for a clear and complete proof
- 2 if boxes etc. are missing, or 1 or 2 wrong rule names are stated while the deductions are correct
- 0 otherwise

Solution

Category (ii): $F_2 \vdash \neg G_2$ is a valid sequent.
Proof using Resolution: We must show that $F_2 \cup \{G_2\}$ is unsatisfiable. First, convert the premises to CNF: $p \to q \equiv \neg p \vee q$, and $q \to r \equiv \neg q \vee r$.

$$\dfrac{\dfrac{\dfrac{\neg p \vee q \quad p}{q} \quad \neg q \vee r}{r} \quad \neg r}{\bot}$$

Marking scheme:

- 3 for a clear and complete resolution proof
- 2 if the proof is correct but do not write all the details of the steps
- 1 if a proof was not given using resolution
- 0 otherwise

Solution

Category (iii): Neither holds (the conclusion is contingent).
To prove this, we provide two truth assignments for the variables (p, q, r):

- **Satisfying Assignment for $F_3 \cup \{G_3\}$:** Set $p = \bot, q = \bot, r = \top$. The premises $(p \to q)$ and $(q \to r)$ evaluate to $\top$. The conclusion $(p \vee r)$ evaluates to $\top$.
- **Falsifying Assignment for $F_3 \cup \{G_3\}$:** Set $p = \bot, q = \bot, r = \bot$. The premises $(p \to q)$ and $(q \to r)$ evaluate to $\top$. However, the conclusion $(p \vee r)$ evaluates to $\bot$. Because we can satisfy the premises while falsifying the conclusion, $F_3 \nvdash G_3$.

Marking scheme:

- +1.5 for a satisfying assignment
- +1.5 for a falsifying assignment

Solution

Category (i): $F_4 \vdash G_4$ is a valid sequent.
Proof using Natural Deduction:

1.	$p \to q$	premise
2.	$q \to r$	premise
3.	$p \vee q$	assumption
4.	p	assumption
5.	q	MP 1, 4
6.	r	MP 2, 5
7.	$q \wedge r$	$\wedge i$ 5, 6
8.	q	assumption
9.	r	MP 2, 8
10.	$q \wedge r$	$\wedge i$ 8, 9
11.	$q \wedge r$	$\vee e$ 3, 4–7, 8–10
12.	$(p \vee q) \to (q \wedge r)$	$\to i$ 3–11

Marking scheme:

- 3 for a clear and complete proof
- 2 if boxes etc. are missing, or 1 or 2 wrong rule names are stated while the deductions are correct. (Please be careful that you don't mix up MP and MT.)
- 0 otherwise

Solution

Category (i): $F_5 \vdash G_5$ is a valid sequent.

1.	$p \to q$	premise
2.	$q \to r$	premise
3.	$q \vee \neg q$	LEM
4.	q	assumption
5.	r	MP 2, 4
6.	$p \vee r$	$\vee i_2$ 5
7.	$(p \vee r) \vee \neg q$	$\vee i_1$ 6
8.	$\neg q$	assumption
9.	$(p \vee r) \vee \neg q$	$\vee i_2$ 8
10.	$(p \vee r) \vee \neg q$	$\vee e$ 3, 4–7, 8–9

Marking scheme:

- 3 for a clear and complete proof
- 2 if boxes etc. are missing, or 1 or 2 wrong rule names are stated while the deductions are correct (we have not deducted marks if someone mixed up MP and MT)
- 2 if someone deduced $\neg q \vee r$ from $q \to r$ directly
- 0 otherwise

3. (8=4+4 marks) Consider the following formula φ,

$$\varphi := (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r) \wedge (\neg p \vee q) \wedge (\neg q \vee r) \wedge (\neg r \vee p)$$

Prove that φ is *unsatisfiable*, using:

(a) resolution,

(b) DPLL algorithm (*without* Pure Literal Elimination). Decisions if necessary should be in the order $\boldsymbol{p, q, r}$.

Solution

(a) Any of the 2 solutions below (either writing step by step, or drawing the tree), done correctly, will give the full 4 marks.

- 0.5 marks are given to everyone who showed at least 1 correct resolution step.
- If, in the proof, it is unclear which 2 clauses the student has taken in the step of resolution, 1 mark is deducted. The exception is if the student has proved by writing the complete $Res^n(F)$ for a step, if this is completely correct, no marks are deducted.
- For each minor mistake ("minor" meaning that despite the mistake, one could still conclude bot correctly), 1 mark is deducted.
- A major mistake would be if the student made the mistake of concluding top (eg $p \vee \neg p$) instead of bot, and didn't proceed further. No marks are given in this case (except the 0.5

which is for showing 1 step of resolution correctly).

- The maximum marks given for using something other than the resolution rule (eg distributive law or a semantic argument) is 2.

(1) $P \vee Q \vee R$ (Premise)

(2) $\neg P \vee \neg Q \vee \neg R$ (Premise)

(3) $\neg P \vee Q$ (Premise)

(4) $\neg Q \vee R$ (Premise)

(5) $\neg R \vee P$ (Premise)

(6) $Q \vee R$ (Resolve 1, 3 on P)

(7) R (Resolve 6, 4 on Q)

(8) $\neg P \vee \neg R$ (Resolve 2, 3 on Q)

(9) $\neg R$ (Resolve 8, 5 on P)

(10) $\bot$ (Resolve 7, 9 on R)

$$\dfrac{\dfrac{\dfrac{P \vee Q \vee R \quad \neg P \vee Q}{Q \vee R} \quad \neg Q \vee R}{R} \qquad \dfrac{\dfrac{\neg P \vee \neg Q \vee \neg R \quad \neg P \vee Q}{\neg P \vee \neg R} \quad \neg R \vee P}{\neg R}}{\bot}$$

Solution

(b) Executing the algorithm either by simplifying step by step, or by drawing the tree, done correctly, will get the full 4 marks.

- 0.5 marks are given if there is at least 1 correct unit propagation step. (No marks are given for enumerating all decisions without using unit propagation anywhere)
- 0.5 marks are deducted for using pure literal elimination.
- 1 mark is deducted every time a decision was taken when a unit propagation was possible. (the deductions due to this is capped at 3 marks - so if someone didn't write the clause on ≥ 3 instances of unit propagation, they lose 3 marks.)
- 1 mark is deducted if the order of taking decisions is not followed.
- 1 mark is deducted each time an assignment is wrong in a unit propagation step
- 1 mark is deducted each time the student concludes a conflict without the proper unit propagation steps. An example would be, deciding P as 0, by unit propagation on C_5, R is 0, and then directly concluding conflict without unit propagation of Q on either C_1 (assign 1) or C_4 (assign 0).

- 0.5 marks are deducted each time a clause is not written in a unit propagation (the deductions due to this is capped at 2 marks - so if someone didn't write the clause on ≥ 4 instances of unit propagation, they lose 2 marks.)
- 0.5 marks are deducted for each wrong conflict clause.
- 2 marks are deducted if the tree is done only for 1 value of the decision variable.

Initial Clause Set Σ:
$\{\{P, Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q\}, \{\neg Q, R\}, \{\neg R, P\}\}$

Executing step by step: *Algorithm Execution:* Finding no UP, we must make a branching decision. Following the priority order, we branch on P.

Branch 1: Decide $P =$ True

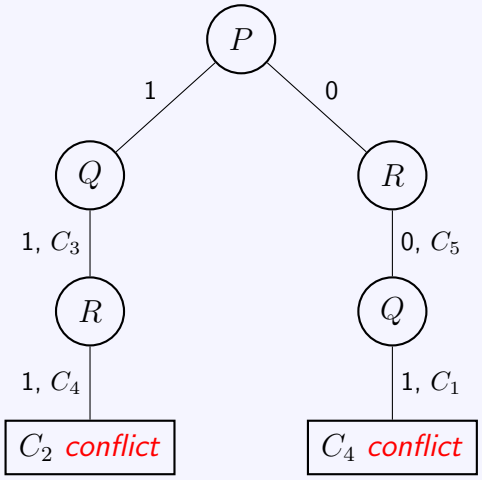
- Clause 1 $\{P, Q, R\}$ is Satisfied.
- Clause 5 $\{\neg R, P\}$ is Satisfied.
- Clause 2 $\{\neg P, \neg Q, \neg R\}$ simplifies to $\{\neg Q, \neg R\}$.
- Clause 3 $\{\neg P, Q\}$ simplifies to $\{Q\}$.
- *Current Clauses:* $\{\{\neg Q, \neg R\}, \{Q\}, \{\neg Q, R\}\}$
- **Unit Propagation on Q:** Set $Q =$ True.
  - Clause 2 $\{\neg Q, \neg R\}$ simplifies to $\{\neg R\}$.
  - Clause 4 $\{\neg Q, R\}$ simplifies to $\{R\}$.
- *Current Clauses:* $\{\{\neg R\}, \{R\}\}$
- **Unit Propagation on R:** Set $R =$ True.
  - Clause $\{\neg R\}$ becomes empty ($\perp$).
- **Conflict!** We derive the empty clause. We must backtrack to the initial state and flip our decision on P.

Branch 2: Decide $P =$ False

- Clause 2 $\{\neg P, \neg Q, \neg R\}$ is Satisfied.
- Clause 3 $\{\neg P, Q\}$ is Satisfied.
- Clause 1 $\{P, Q, R\}$ simplifies to $\{Q, R\}$.
- Clause 5 $\{\neg R, P\}$ simplifies to $\{\neg R\}$.
- *Current Clauses:* $\{\{Q, R\}, \{\neg Q, R\}, \{\neg R\}\}$
- **Unit Propagation on $\neg R$:** Set $R =$ False.
  - Clause 1 $\{Q, R\}$ simplifies to $\{Q\}$.
  - Clause 4 $\{\neg Q, R\}$ simplifies to $\{\neg Q\}$.

- *Current Clauses:* $\{\{Q\}, \{\neg Q\}\}$
- **Unit Propagation on Q:** Set $Q = \text{True}$.
  - Clause $\{\neg Q\}$ becomes empty ($\bot$).
- **Conflict!** We derive the empty clause on this branch as well.

Conclusion: Both the $P = \text{True}$ and $P = \text{False}$ branches deterministically lead to a contradiction ($\square$) through Unit Propagation. Therefore, the set of clauses is **Unsatisfiable**.



Tree Explanation:

- **Left Branch ($P = 1$):** The decision $P = 1$ forces $Q = 1$ via clause C_3. Setting $Q = 1$ forces $R = 1$ via clause C_4. This assignment $(P = 1, Q = 1, R = 1)$ causes a conflict in clause C_2.
- **Right Branch ($P = 0$):** The decision $P = 0$ forces $R = 0$ via clause C_5. Setting $R = 0$ forces $Q = 1$ via clause C_1. This assignment $(P = 0, R = 0, Q = 1)$ causes a conflict in clause C_4.

Common mistakes

- **Part (a): Resolution (4 marks)**
  - Concluding top (eg $p \vee \neg p$) and thinking the proof is over.
- **Part (b): DPLL Search Tree (4 marks)**
  - Concluding the conflict without doing the final unit propagation step
  - Typos in writing the clauses for unit propagation

4. (6 marks) Prove that the below set of 6 clauses is unsatisfiable using the CDCL algorithm, make sure to draw the implication graphs, and identify any conflict clause clearly.

$C_1 : \neg P \vee \neg Q \vee R$
$C_2 : \neg P \vee \neg Q \vee \neg R$
$C_3 : \neg P \vee Q \vee R$
$C_4 : \neg P \vee Q \vee \neg R$
$C_5 : P \vee Q$
$C_6 : P \vee \neg Q$

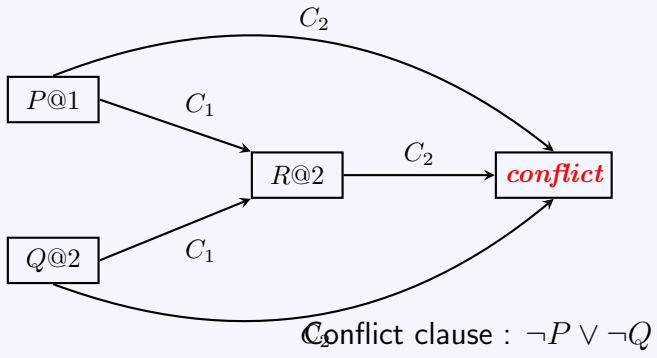
You must use the following **CDCL Rules for this problem:**

- **Decision:** Decisions must be in the variable order P, Q, R. When making a branching decision on a variable, you must always assign it the value **True (you cannot decide False)**.
- **Clause Learning:** When a conflict occurs, traverse the implication graph backwards to find the set of decision nodes that caused the conflict. The **learned conflict clause** is the disjunction of the negations of these causing decisions.
- **Backtracking:** Backjump to the *second highest* decision level present in the learned clause. If the learned clause is a unit clause, backjump all the way to **Level 0**.

Solution

Decision Levels 1 & 2 (First Conflict)

- **Level 0:** No unit clauses.
- **Level 1:** Decide $P = \text{True}$ ($P@1$). No unit propagation is triggered yet.
- **Level 2:** Decide $Q = \text{True}$ ($Q@2$).
- **Unit Propagation:**
  - C_1 ($\neg P \vee \neg Q \vee R$) forces $R@2$.
- **Conflict!** Clause C_2 ($\neg P \vee \neg Q \vee \neg R$) is violated.

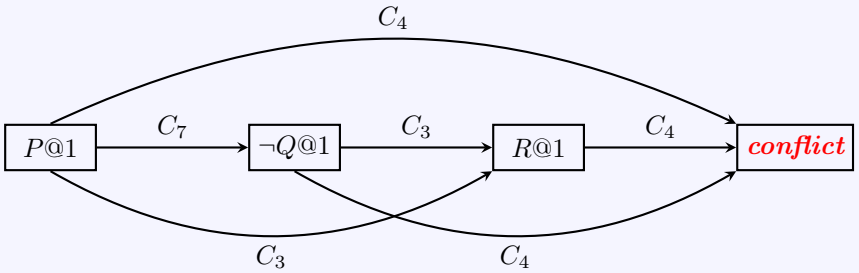


Explanation: Traversing backwards, the conflict is caused by the decisions $P@1$ and $Q@2$. Negating them yields the learned clause $C_7 = \neg P \vee \neg Q$. The second highest decision level in this clause is Level 1, so we **backjump to Level 1**.
Marking Scheme: First Conflict & Backjump (2 marks):

- **0.5 marks** for the correct sequence of decisions ($P@1, Q@2$) and initial unit propagation ($R@2$).
- **1.0 mark** for a perfectly drawn Implication Graph 1 (correct nodes, directed edges annotated with clauses, and the conflict node).
- **0.5 marks** for correctly extracting the learned clause ($\neg P \vee \neg Q$) and correctly identifying the backjump to Level 1.

Decision Level 1 (After First Backjump)

- We retain decision $P@1$ and add learned clause C_7.
- **Unit Propagation:**
  - C_7 ($\neg P \vee \neg Q$) forces $\neg Q@1$.
  - C_3 ($\neg P \vee Q \vee R$) now has $P = 1, Q = 0$, forcing $R@1$.
- **Conflict!** Clause C_4 ($\neg P \vee Q \vee \neg R$) is violated.



Conflict clause : $\neg P$

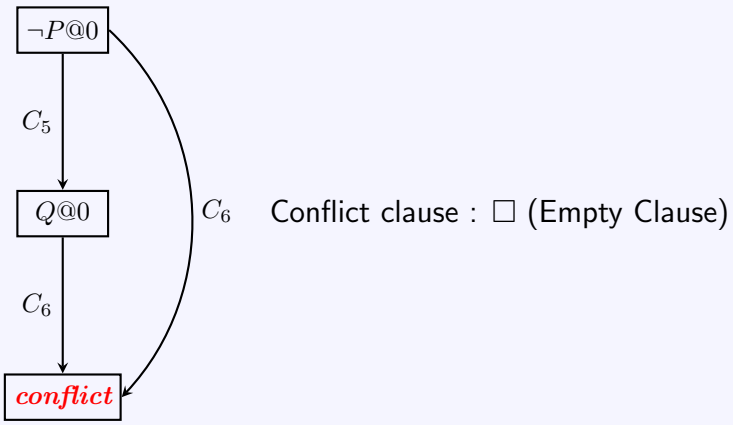
Explanation: Traversing backwards from the conflict, all paths eventually lead to a single root decision node: $P@1$. Negating it yields the learned clause $C_8 = \neg P$. Because this is a unit clause, we **backjump to Level 0**.

Marking Scheme - Second Conflict & Backjump (2 marks):

- **0.5 marks** for correct unit propagations at Level 1 ($\neg Q@1$, $R@1$).
- **1.0 mark** for a perfectly drawn Implication Graph 2.
- **0.5 marks** for correctly extracting the unit clause ($\neg P$) and correctly identifying the backjump to Level 0.

Decision Level 0 (After Second Backjump)

- We add learned clause $C_8 = \neg P$ to our formula.
- **Unit Propagation:**
  - C_8 forces $\neg P@0$.
  - C_5 $(P \vee Q)$ forces $Q@0$.
- **Conflict!** Clause C_6 $(P \vee \neg Q)$ is violated.



Explanation: We traverse backwards to find the causing decisions. Because this entire cascade happened at Level 0, there are **no** decision nodes in this graph (the root node $\neg P$ was forced, not decided). The set of causing decisions is empty, making the conflict clause the empty clause ($\perp$). The formula is **Unsatisfiable**.

Marking Scheme - Level 0 Conflict & Conclusion (2 marks):

- **0.5 marks** for correct unit propagations at Level 0 ($\neg P@0$, $Q@0$).
- **1.0 mark** for a perfectly drawn Implication Graph 3.
- **0.5 marks** for explicitly deriving the empty clause ($\square$) and correctly concluding that the formula is UNSAT.

Extras

- Deduct an automatic **1.0 mark** overall if the student branched on **False** or disobeyed the **P, Q, R** variable order at any point.
- Deduct **0.5 mark** in each implication graph if the clauses are not written on the edges.
- Deduct **0.5 mark** in each implication graph if the last node before conflict node is missing.
- Deduct **0.5 mark** in each implication graph if any variable is assigned both True and False in 2 nodes of the same graph.
- Deduct **1 mark** in each of 2nd and 3rd implication graphs if the learnt clause from the previous graph is wrong and the concerned implication graph is correct as per CDCL.
- Deduct **0.5 mark** if Empty Clause is not derived in 3rd implication graph.

- Deduct **0.5 mark** in each implication graph if Decision Level is not mentioned or written wrong.
- **Upto 1 mark** is given in each of the 3 runs shown using **only** DPLL **iff** the run is correct and the learnt clause for that run is correct.

---

5. (14=5+3+5+1 marks) Recall that a k-CNF formula has at most k literals in every clause.
   (a) Show that we can convert any k-CNF formula into an equi-satisfiable 3-CNF formula. Recall that φ, ψ are equi-satisfiable if φ is satisfiable iff ψ is satisfiable.
   (b) Does this conversion preserve equivalence? Why or why not? Recall that φ, ψ are equivalent if their truth tables are identical.
   (c) Is there any such conversion that preserves equivalence? That is, does there exist a k-CNF formula ($k \geq 4$) for which there is no equivalent 3-CNF? If yes, show such an example. If not, prove it by giving an equivalence preserving conversion.
   (d) Do you think there exists a polynomial-time conversion from any k-CNF formula to an equisatisfiable 2-CNF formula? What would be the consequence of such a conversion? You may assume any result discussed in class or in the problemsheets to answer this.

Solution

[a] Let F be a k-CNF formula, i.e. every clause of F has exactly k literals. We build an equisatisfiable 3-CNF formula F' by replacing each clause of F independently (using a disjoint set of fresh auxiliary variables per clause) and conjoining the results.
Case $k \leq 3$: pad the clause to exactly 3 literals by repeating one of its literals as many times as needed (e.g. $\ell_1 \equiv \ell_1 \vee \ell_1 \vee \ell_1$, or $(\ell_1 \vee \ell_2) \equiv (\ell_1 \vee \ell_2 \vee \ell_2)$). This changes nothing semantically – the padded clause is in fact logically *equivalent* to the original, not merely equisatisfiable, since no new variables are introduced.
Case $k \geq 4$: let $C = (\ell_1 \vee \ell_2 \vee \cdots \vee \ell_k)$ be a clause of F. Introduce $k-3$ fresh variables $y_1, \ldots, y_{k-3}$, not occurring elsewhere in the formula, and replace C by

$$C' \;:=\; (\ell_1 \vee \ell_2 \vee y_1) \;\wedge\; \bigwedge_{i=1}^{k-4} (\neg y_i \vee \ell_{i+2} \vee y_{i+1}) \;\wedge\; (\neg y_{k-3} \vee \ell_{k-1} \vee \ell_k),$$

a conjunction of clauses each with exactly 3 literals: the "entry" clause pulls in ℓ_1, ℓ_2; each "middle" clause pulls in one more literal ℓ_{i+2}, chained to the next via y_i, y_{i+1}; the "exit" clause pulls in the last two literals ℓ_{k-1}, ℓ_k. (1 mark)
Proof that C is satisfiable iff C' is:
($\Rightarrow$) (2 marks) Suppose some literal ℓ_i of C is true under an assignment. Extend it to the y_j's by a "cascade": set $y_j = \top$ for $j \leq i-2$ and $y_j = \bot$ for $j \geq i-1$ (or all $y_j = \top$ if $i \in \{k-1, k\}$). One checks directly that every conjunct of C' is then satisfied: each entry/middle/exit clause is satisfied either directly by ℓ_i, or because the relevant y_j (or $\neg y_j$) has been switched on by the cascade. Hence C' is satisfiable.
($\Leftarrow$) (2 marks) Suppose, for contradiction, C' is satisfied by some assignment under which *every* ℓ_i is false. The entry clause forces $y_1 = \top$ (since ℓ_1, ℓ_2 are false). Inductively, if $y_j = \top$, the middle clause $(\neg y_j \vee \ell_{j+2} \vee y_{j+1})$ has $\neg y_j$ and ℓ_{j+2} both false, forcing $y_{j+1} = \top$; so $y_{k-3} = \top$. But then the exit clause $(\neg y_{k-3} \vee \ell_{k-1} \vee \ell_k)$ has all three literals false, contradicting that the assignment satisfies C'. So some ℓ_i must be true, i.e. C is satisfied.
Applying this clause-by-clause (fresh auxiliary variables per clause, so different clauses' replace-

ments cannot interact) converts F into a 3-CNF formula F', with F satisfiable iff every clause's replacement is simultaneously satisfiable iff F' is satisfiable. Hence F' is equisatisfiable to F.

Alterntate solution using Tseitin Encoding

Let F by any k-CNF formula. Now we show that appying Tseitin Encoding (1 mark) on F, we get a 3-CNF. Since we only have $\vee$ and $\wedge$ operators in F, we must prove that both produce clauses with atmost 3 literals.

Case 1: $a \iff b \vee c$
This encoding produces the clauses: $\neg a \vee b \vee c$, $\neg b \vee a, \neg c \vee a$ Which all have ≤ 3 literals (1.5 marks)

Case 2: $a \iff b \wedge c$ This encoding produces the clauses: $\neg b \vee \neg c \vee a$,$\neg a \vee b$, $\neg a \vee c$ which all have ≤ 3 literals (1.5 marks)
From the slides, we know Tseitin preserves equisatisfiability. Hence we have made an equisatisfiable CNF formula where each clause has ≤ 3 literals. (1 mark)
Draft Marking Scheme

Some people have used Induction for which the rubrics are:

2 marks if the construction produced by the Induction is an Equisat 3-CNF
1.5 marks each for $\vee$ and $\wedge$ case.

Common Mistakes:

(a) Many cases only proved a per clause $\vee$ case and didn't do the $\wedge$ case which is not the complete proof. Eg. q is equi-sat to p and q is equi-sat to $\neg$ p does not imply that q $\wedge \neg$ q is equi-sat to p $\wedge \neg$ p.

(b) for induction many people started the base case as k= 0 or k=1. That is wrong. Base case should include 3 otherwise induction goes wrong. 0.5 marks dedected

Solution

[b] No, this conversion does not preserve logical equivalence in general (only for the trivial $k \leq 3$ padding case, which introduces no new variables). (1 mark)
(2 marks) Counterexample: take $k = 4$, $C = \ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4$, so $C' = (\ell_1 \vee \ell_2 \vee y_1) \wedge (\neg y_1 \vee \ell_3 \vee \ell_4)$. Under the assignment $\ell_1 = \top$, $\ell_2 = \ell_3 = \ell_4 = \bot$, $y_1 = \top$: $C = \top$, but in C' the first clause is true while the second clause $(\neg y_1 \vee \ell_3 \vee \ell_4) = (\bot \vee \bot \vee \bot) = \bot$, so $C' = \bot$. They disagree, hence are not equivalent.
More generally: C does not mention the y_j's, so its truth value is constant across all choices of the y_j's, whereas C' genuinely depends on them. So for a fixed satisfying assignment of C, there is always some "bad" choice of the y_j's making C' false. Equisatisfiability only asks that *some* assignment to the y_j's make C' true whenever C is true (exactly the cascading choice built in part (a)) – not that *every* assignment does, which is what equivalence would require. This is why introducing fresh variables can only ever preserve satisfiability, never give a logically equivalent formula.

Counterexample for Tseitin
Consider $F = (\ell_1 \vee \ell_2)$, after applying Tseitin we get, $(a) \vee (\neg a \vee \ell_1 \vee \ell_2) \vee (\neg \ell_1 \vee a) \vee (\neg \ell_2 \vee a)$

In the assignment $a = \bot, \ell_1 = \top, \ell_2 = \top$, the original formula evaluates to $\top$ and the encoded formula evaluates to $\bot$
Note: Any arguments based on number of variables/length of truth table is incorrect as equivalence is considered over union of propositional vars of both formulae.
The 1 mark for saying no is only given if there is some correct reasoning in either part a) or c)

Solution

[c] Not in general (1 mark): for $k \geq 4$ there exist k-CNF formulas with no logically equivalent 3-CNF formula at all, so equivalence can't be preserved in general. (For $k \leq 3$, the padding of part (a) trivially works.)
(1 mark) First, we show that if there is an equivalent formula in 3-CNF with additional variables included, there is also an equivalent formula with only the original variables, and hence we will only need to disprove this case.
Consider a equivalent 3-CNF formula with additional variables. Since this is equivalent, for every assignment to the original variables the truth value of the original formula is the same as the new one, irrespective of the assignments to the new variables.
Hence if we assign all the new variables to true, we get another equivalent formula as the truth values are still the same. Since this formula is also in 3-CNF over only the original variables, we are done
(1 mark) Now take F to be the single clause $C = \ell_1 \vee \cdots \vee \ell_k$ (itself a valid k-CNF formula). We show no 3-CNF formula over just $\ell_1, \ldots, \ell_k$ is equivalent to C, by a counting argument on falsifying assignments.
(2 marks) C is false under *exactly one* assignment: $\ell_1 = \cdots = \ell_k = \bot$ (every other assignment makes some ℓ_i true). So any formula equivalent to C must be false at exactly that one point, and true everywhere else.
Suppose $G = C_1 \wedge \cdots \wedge C_m$ is a CNF formula over $\ell_1, \ldots, \ell_k$ only, with each C_j having at most 3 literals. Each C_j mentions at most 3 of the k variables, leaving $\geq k - 3 \geq 1$ variables completely unconstrained by C_j. Fixing the (at most 3) variables of C_j to falsify all its literals and letting the rest range freely gives $\geq 2^{k-3} \geq 2$ distinct falsifying assignments for C_j alone – hence for G (since G is false whenever any conjunct is). This set of size ≥ 2 can never equal the single-point falsifying set required for $G \equiv C$, so $G \not\equiv C$ (and the empty conjunction $\top$, i.e. $m = 0$, is trivially not equivalent either).
So bounding clause width to 3 while staying within the original variables is impossible without changing the formula's models – new variables are unavoidable, which is exactly why equivalence (as opposed to mere equisatisfiability) cannot be preserved in general.
Note: If you have given a general argument(and not mentioned anything about only applying it on existing variables) which works for both fresh variables and no-fresh variables then we have given marks.

Solution

[d] **No** (assuming P $\neq$ NP). It is a known result that 3-SAT is NP-complete, whereas 2-SAT can be solved in polynomial time (i.e., 2-SAT $\in$ P). If there existed a polynomial-time conversion from any k-CNF (including 3-CNF) to an equisatisfiable 2-CNF formula, we could solve 3-SAT in polynomial time by first converting it, and then running a polynomial-time 2-SAT solver. This would reduce an NP-complete problem to P, implying P = NP.

We can solve 2-SAT in polynomial (specifically, linear) time which you will learn in DSA!

1 mark for correct reasoning using NP completeness of 3-SAT

6. (20=6+5+5+4 marks) Recall that a formula is said to be in Negation Normal Form (NNF) if negation is only on the leaves (which can be a variable, $\top$, or $\bot$) and all internal nodes are $\wedge$ and $\vee$. For a node u, write $\text{Vars}(u)$ for the set of variables occurring (positively or negatively) among the literals in the sub-tree rooted at u – e.g. $\text{Vars}(\ell) = \{x\}$ for a literal ℓ over variable x, and $\text{Vars}(\top) = \text{Vars}(\bot) = \emptyset$.

   - An NNF is *decomposable* if, for every $\wedge$-node with children $c_1, \ldots, c_k$, the sets of variables $\text{Vars}(c_1), \ldots, \text{Vars}(c_k)$ are pairwise disjoint.
   - An NNF is *deterministic* if, for every $\vee$-node with children $c_1, \ldots, c_k$, the c_i are pairwise logically inconsistent (i.e. $c_i \wedge c_j$ is unsatisfiable for all $i \neq j$ – equivalently, at most one child can be true under any given assignment).
   - An NNF is said to be in **d-DNNF** (*deterministic decomposable NNF*) if it is both decomposable and deterministic.

   As an **example,** the formula $\varphi = (p \wedge q) \vee (\neg p \wedge r)$ is in d-DNNF: each $\wedge$-node's two children are single literals over distinct variables ($\{p\}, \{q\}$, and $\{p\}, \{r\}$ respectively), so both are decomposable; and the two disjuncts $(p \wedge q)$, $(\neg p \wedge r)$ disagree on p and so are pairwise logically inconsistent, making the $\vee$-node deterministic.

   (a) For each of the following, state whether it is **TRUE** or **FALSE**, and justify your answer.
      (i) A formula can be decomposable but not deterministic.
      (ii) A formula can be deterministic but not decomposable.
      (iii) If φ is decomposable, then φ is in DNF.
      (iv) If φ is in DNF, then φ must be decomposable.

   (b) Give an (as efficient as possible) algorithm to convert *any* propositional formula into an equivalent d-DNNF. In the worst case, how much is the blowup in the size of the d-DNNF that you construct using your algorithm?

   (c) Consider the problem of *model counting*, which asks to count the number of satisfying assignments (models) of a formula. Given a formula in d-DNNF, prove or disprove that model counting can be solved in polynomial time in the size of the formula. (The size of the formula here is just the sum of the number of nodes and edges in its parse tree.)

   (d) Using the above ideas, can you come up with a direct (and efficient) DPLL-style algorithm for model counting (for general CNF formulas)?

Solution

[a] **(i) TRUE.** Take $\psi_1 = (p \wedge q) \vee (p \wedge r)$. Each $\wedge$-node's children are single literals over distinct variables, so ψ_1 is decomposable. But the two disjuncts are *not* mutually inconsistent: the assignment $p = q = r = \top$ satisfies both $(p \wedge q)$ and $(p \wedge r)$ simultaneously, so the $\vee$-node is not deterministic. (In fact ψ_1 is a DNF formula, and part (iv) below shows every DNF formula is automatically decomposable – so DNF formulas are the generic source of "decomposable but not deterministic" examples, since two DNF terms need not be mutually exclusive.)
(ii) TRUE. Take $\psi_2 = (p \wedge p) \vee \neg p$. The $\wedge$-node's two children are both the literal p, sharing the variable $\{p\}$ – *not* disjoint, so ψ_2 is not decomposable. But the $\vee$-node's children, $(p \wedge p)$ (equivalent to p) and $\neg p$, are mutually inconsistent (one requires p, the other $\neg p$), so the $\vee$-node is deterministic. This illustrates that decomposability and determinism are genuinely independent syntactic conditions, imposed on different node types ($\wedge$ vs. $\vee$) – and, more subtly, that they are properties of the *circuit* representing a formula, not of the Boolean function it computes: ψ_2 is

semantically just the tautology $p \vee \neg p$, yet this particular circuit for it fails decomposability.
(iii) FALSE. Take $\varphi = p \wedge (q \vee r)$. Its single $\wedge$-node has children p and $(q \vee r)$, with disjoint variable sets $\{p\}$ and $\{q, r\}$, so φ is decomposable. But φ is not in DNF: DNF requires a flat disjunction of conjunctions of literals, with no $\vee$ nested inside an $\wedge$, whereas here an $\vee$-node is a child of an $\wedge$-node. (Distributing to get an equivalent DNF, $(p \wedge q) \vee (p \wedge r)$, requires an entirely different circuit.)
(iv) TRUE, under the standard convention – but FALSE if read without it. A DNF formula is $\bigvee_i \bigwedge_j \ell_{ij}$: a disjunction of terms, each term a conjunction of literals. Read completely literally, with no restriction on the literals inside a term, this is *false*: $\varphi = (p \wedge \neg p) \vee q$ is a syntactically valid DNF formula, but its first term's $\wedge$-node has children p and $\neg p$, both over $\{p\}$ – not disjoint, so it is not decomposable.
The intended reading, however – standard in the knowledge-compilation literature and the one meant here – takes a (non-degenerate) DNF term to be a conjunction of literals over *pairwise distinct* variables (no variable repeated, and in particular no term containing both a variable and its negation). Under this convention the claim is *true*: each term $\ell_1 \wedge \cdots \wedge \ell_k$ has literals over pairwise distinct variables $x_1, \ldots, x_k$ by assumption, so as an $\wedge$-node its children have singleton, pairwise disjoint variable sets $\{x_1\}, \ldots, \{x_k\}$ – decomposable. Since every $\wedge$-node in a DNF circuit is exactly one such term (the only other node is the top-level $\vee$, which decomposability does not constrain), the whole formula is decomposable.
0.5 marks per part for True/False, 1 mark per part for correct example/explanation for parts (i), (ii), (iii).
1.5 marks for True/False with correct justification for (iv). 0.5 marks deduction for minor error in justification. 0 marks for part (iv) if justification is incorrect.
Common mistakes:

- Many people gave examples which were not valid counter-examples.
- Many have argued that a formula ϕ is in CNF and hence not DNF. This is wrong since a formula can be in both CNF and DNF. Marks have been awarded for correct counter-example.

Solution

[b] **Yes.** Every propositional formula φ over variables $x_1, \ldots, x_n$ can be converted to an equivalent d-DNNF by repeated Shannon expansion (case-splitting on one variable at a time), stopping once every remaining sub-formula is a constant:

$$\varphi \;\equiv\; (x_1 \wedge \varphi|_{x_1=\top}) \;\vee\; (\neg x_1 \wedge \varphi|_{x_1=\bot}),$$

and recursively expand $\varphi|_{x_1=\top}$ and $\varphi|_{x_1=\bot}$ on x_2, and so on, until no variables remain and each leaf sub-formula has collapsed to $\top$ or $\bot$.
This construction is a d-DNNF, by induction on the recursion. At the base case, a constant $\top$ or $\bot$ is trivially both decomposable and deterministic (it has no $\wedge$- or $\vee$-nodes at all). For the inductive step, suppose the circuits built for $\varphi|_{x_i=\top}$ and $\varphi|_{x_i=\bot}$ are already d-DNNFs (over the remaining variables $x_{i+1}, \ldots, x_n$, since $x_1, \ldots, x_i$ have all been substituted away and no longer occur). The new $\wedge$-nodes are $(x_i \wedge \varphi|_{x_i=\top})$ and $(\neg x_i \wedge \varphi|_{x_i=\bot})$: in each, one child is the single variable $\{x_i\}$ and the other child's sub-circuit does not mention x_i at all – disjoint, so decomposable. The new $\vee$-node combines these two: one branch forces $x_i = \top$, the other forces $x_i = \bot$, so they are mutually inconsistent – deterministic. Hence the whole new circuit is again a d-DNNF, completing the induction.
Caveat. This always succeeds, but the resulting circuit can be as large as the full binary expansion

tree, i.e. $\Theta(2^n)$ in the worst case – so while a d-DNNF *always exists*, converting to it is not generally a polynomial-time (or polynomial-size) operation; some formulas are known to require d-DNNFs of exponential size. This is the basic trade-off of *knowledge compilation*: the (possibly expensive) one-time compilation into d-DNNF is what buys the polynomial-time queries proved in parts (c) and (d) below.

2 marks for a correct efficient construction, 2 marks for explanation of construction/its correctness, 1 mark for complexity calculations

Common mistakes:

- Some students gave a solution based on checking the truth table and constructing clauses accordingly. While this approach is correct, it has a complexity of $\Omega(2^n)$, and therefore a maximum of 3.5 marks was awarded for this approach.
- If the solution provided by the student was correct and had better performance than the algorithm presented in our solution for some cases, full marks were awarded.

Solution

[c] Fix the variable set $X = \mathrm{Vars}(\varphi)$ of size n. For a node u, let $\mathrm{Vars}(u) \subseteq X$ be the variables occurring in u's sub-circuit, and define

$$\mathrm{count}(u) \;=\; \#\{\text{assignments to } \mathrm{Vars}(u) \text{ that satisfy } u\},$$

computed bottom-up as follows.

Leaves. $\mathrm{count}(\top) = 1$ (the empty assignment satisfies it), $\mathrm{count}(\bot) = 0$; for a literal ℓ over variable x, $\mathrm{Vars}(\ell) = \{x\}$ and $\mathrm{count}(\ell) = 1$ (exactly one of the two assignments to $\{x\}$ makes ℓ true).

$\wedge$-nodes (uses decomposability). With children $c_1, \ldots, c_k$ over pairwise disjoint $V_1, \ldots, V_k$ (so $\mathrm{Vars}(\wedge) = \bigcup_i V_i$): by exactly the merge/restrict argument from part (c) – disjointness means satisfying assignments to each V_i combine freely and independently, with no shared variable to create a dependency between them – the number of joint satisfying assignments is the product:

$$\mathrm{count}(\wedge\text{-node}) \;=\; \prod_i \mathrm{count}(c_i).$$

$\vee$-nodes (uses determinism). Let $c_1, \ldots, c_k$ be the children, with $\mathrm{Vars}(\vee) = \bigcup_i \mathrm{Vars}(c_i)$ (the $\mathrm{Vars}(c_i)$ need not be disjoint here). First extend each child's count to the full set $\mathrm{Vars}(\vee)$: since c_i places no constraint on a variable $y \in \mathrm{Vars}(\vee) \setminus \mathrm{Vars}(c_i)$ it doesn't mention, each such variable doubles the count freely, giving an extended count $\mathrm{count}(c_i) \cdot 2^{|\mathrm{Vars}(\vee) \setminus \mathrm{Vars}(c_i)|}$ for the number of assignments to $\mathrm{Vars}(\vee)$ satisfying c_i. Determinism says $c_i \wedge c_j$ is unsatisfiable for $i \neq j$, so no assignment (to any superset of variables, in particular to $\mathrm{Vars}(\vee)$) can satisfy two children at once – the children's satisfying sets, viewed over $\mathrm{Vars}(\vee)$, are pairwise *disjoint* subsets of the $2^{|\mathrm{Vars}(\vee)|}$ possible assignments. The count of their union is therefore exactly the sum of the (extended) individual counts, with no double-counting:

$$\mathrm{count}(\vee\text{-node}) \;=\; \sum_i \mathrm{count}(c_i) \cdot 2^{|\mathrm{Vars}(\vee) \setminus \mathrm{Vars}(c_i)|}.$$

(If the circuit happens to be *smooth* – every $\vee$-node's children share exactly the same variable set – this correction factor is always 1 and the rule is just a plain sum.)

Correctness and complexity. By induction on the circuit (children before parents, as in part (c)), $\mathrm{count}(u)$ is correctly computed at every node, so $\mathrm{count}(\varphi)$ (at the root) is the true number

of models of φ. Every node performs $O(1)$ multiplications/additions of integers with at most n bits, so using schoolbook arithmetic each node costs $O(\text{poly}(n))$ time; summed over all nodes this is $O(\text{poly}(n) \cdot \text{size of circuit})$ – polynomial in the size of the d-DNNF (and in n). □
0.5 marks for an idea of tree traversal, 1 mark for the correct step at and nodes, 1 mark for the correct step at or nodes, 2 marks for the induction argument and 0.5 marks for the time complexity.
Common mistakes:

- Writing that the count of a $\vee$ node is the direct sum of counts of the children of the node.
- Forgetting to do the time complexity analysis of their algorithm.

Solution

[d] **Yes.** We can construct an efficient DPLL-style model counting algorithm, often referred to as **Counting DPLL (#DPLL)** or **CDP**.

Algorithm 1 CDP (F, t)

Input: A CNF formula F over n variables; a parameter t initially set to 0
Output: $\#F$, the model count of F
`UnitPropagate`(F)
if *F has an empty clause* **then**
    return 0
end if
if *all clauses of F are satisfied* **then**
    return 2^{n-t}
end if
$x \leftarrow$ `SelectBranchVariable`(F)
return `CDP`$(F|_x, t+1) +$ `CDP`$(F|_{\neg x}, t+1)$

Algorithm Explanation:

- **The Parameter t:** The variable t tracks the total number of variables that have been assigned a truth value so far in the current recursive branch.
- **Base Case 1 (Conflict):** If unit propagation or a previous decision results in an empty clause, the current partial assignment cannot satisfy the formula. The algorithm returns 0 for this branch.
- **Base Case 2 (Early Success & Don't Cares):** If the current partial assignment satisfies all clauses in F, the algorithm does not need to explicitly branch on the remaining unassigned variables. Since t variables have been assigned, there are exactly $n - t$ unassigned variables left. Because the formula is already satisfied, these remaining variables are "don't cares"—they can be assigned True or False freely without changing the outcome. The algorithm efficiently counts all these possible completions at once by returning 2^{n-t}.
- **Recursive Step (Branching):** If the formula is neither fully satisfied nor falsified, the algorithm selects an unassigned variable x. It recursively calls itself twice: once with x assigned True ($F|_x$) and once with x assigned False ($F|_{\neg x}$). The total model count is the

strict sum of the counts from both branches, ensuring no double-counting occurs because the branches are mutually exclusive.

2.5 marks for a pseudocde/description of the algorithm (only if it is similar to DPLL), 1.5 marks for explanation of why it produces the correct output

Common mistakes:

- Some students have used the conversion of part (b) to convert the formula to a d-DNNF and then using some model counting algorithm. This has not been awarded any marks. The problem clearly asks for a "DPLL-style" algorithm.

---

7. (1=0.5+0.5 marks) Write the full form of DPLL. We have now seen two proof systems for propositional logic, namely, natural deduction and resolution. Which do you like better and why? Give a 1-2 sentence argument.

Solution

Full form of DPLL: Davis-Putnam-Logemann-Loveland (algorithm).
0.5 marks for any two of the four names are about right.

Preference (Answers will vary; award 0.5 marks for any valid 1-2 sentence justification):

- *Resolution:* I prefer Resolution because it relies on a single rule.